MODULE 03 · DOMAIN 1 - MONITORING, LOGGING, ANALYSIS, REMEDIATION, AND PERFORMANCE OPTIMIZATION

CloudWatch Logs and the Unified Agent

Getting logs off EC2, ECS and EKS, and keeping the bill under control once they arrive.

DOMAIN 1 of 5 EXAM WEIGHT 22% MODULE 3 of 35

admin@sthithapragna.com

The agent configuration is a **deployment artefact**, not a file on a server. Treat it as a file and it drifts within a week.

The log data model

- **Log event.** One record with a timestamp. **Log stream.** A sequence from one source, usually one instance or container.
- **Log group.** The unit of everything that matters: retention, encryption, access, metric filters, subscriptions, log class.
- **Retention defaults to Never expire.** Nobody chooses this. It is what you get by not choosing.
- Up to **1,000,000 log groups** per Region per account, so the limit is your bill, not the service.

> ON THE EXAM
>
> Every setting you care about lives on the log group, not the stream. Streams are just where the lines land.

Retention, and the bill

SETTING	WHAT IT MEANS IN PRACTICE
`Never expire (the default)`	You pay storage on every line, for ever
`1 to 3653 days`	Fixed steps only: 1, 3, 5, 7, 14, 30, 60, 90 … up to 10 years
`Set on the log group`	Applies to every stream in it. Expired events are deleted automatically
`Ingestion vs storage`	Two separate charges. Retention only reduces the second one
`Export to S3`	For cheap long-term retention. S3 lifecycle rules then apply

ON THE EXAM

Retention is a fixed list, not any number you like. "Set retention to 45 days" is not a valid option, and the exam knows it.

What the unified agent actually is

- **One binary that does both jobs:** system metrics and log files. It replaced the old separate awslogs agent and collectd setup.
- Collects **memory, disk, swap, processes** - everything module one said the hypervisor cannot see.
- Runs on **EC2, on-premises servers and other clouds**. The same agent, with a different mode flag.
- Also accepts **StatsD and collectd** custom metrics, and OTLP metrics and traces.

> ON THE EXAM
>
> "Unified" means metrics and logs in one agent. If a scenario mentions two agents, it is describing something that was retired.

Deploying it so it stays deployed

01

Install

SSM Distributor, or bake it into the AMI.

02

Attach a role

CloudWatchAgentServerPolicy on the instance profile.

03

Store the config

Put the JSON in SSM Parameter Store, once.

04

Push it

Run Command to apply now, State Manager to keep it applied.

05

Verify

Check the log group appears and the agent's own log for errors.

ON THE EXAM

The config lives in Parameter Store and the agent fetches it with an ssm: prefix. That is the whole pattern, and the exam expects it.

Run Command or State Manager?

RUN COMMAND WHEN

Do it once, now

- A one-off change across a fleet
- You want the result immediately
- Applies to instances running at that moment
- Nothing re-applies it afterwards

STATE MANAGER WHEN

Keep it true

- A configuration that must not drift
- New instances get it automatically
- Re-applies on a schedule you set
- Someone editing the box by hand gets corrected

THE TIEBREAKER

"Instances launched later must also be configured" or "the setting keeps getting changed" both mean State Manager.

The agent is installed but nothing appears

- **Is there an instance profile** with CloudWatchAgentServerPolicy? The most common cause by a distance.
- **Can it reach the endpoints?** A private subnet with no NAT and no VPC endpoints for logs and monitoring will fail silently.
- **Is the config actually applied?** Installed is not configured. The agent starts with nothing to collect.
- **Read the agent's own log.** /opt/aws/amazon-cloudwatch-agent/logs on Linux. It says exactly what is wrong.

> ON THE EXAM
>
> All four failures look identical from the console: an empty log group, or no log group at all. Work the list in order.

Standard or Infrequent Access log class?

STANDARD WHEN

The logs do work

- Metric filters, subscription filters
- Live Tail, log anomaly detection
- Real-time alerting off log content
- Higher ingestion price per GB

INFREQUENT ACCESS WHEN

The logs are evidence

- Forensics, audit, occasional investigation
- Lower ingestion price per GB
- Logs Insights, export to S3, encryption
- March 2026: masking, OpenSearch PPL and SQL

THE TIEBREAKER

Only ingestion price differs. Storage and Logs Insights query charges are identical in both classes.

Log class rules worth memorising

- **The class cannot be changed after the log group is created.** Wrong choice means a new log group.
- Infrequent Access has **no metric filters, no subscription filters, no Live Tail, no anomaly detection**.
- It **does** have Logs Insights, export to S3, encryption and cross-account analytics.
- **Updated March 2026:** Infrequent Access gained **data protection masking** plus OpenSearch PPL and SQL queries.

> ON THE EXAM
>
> If a scenario needs an alarm on something in the log, the group must be Standard. Metric filters and subscription filters are the deciding capabilities.

ECS: awslogs driver or FireLens?

AWSLOGS DRIVER

Straight to CloudWatch Logs

- Set in the task definition's logConfiguration
- Simple, no extra container to run
- One destination: CloudWatch Logs
- The default answer for ECS container logs

FIRELENS

A log router as a sidecar

- Runs Fluent Bit or Fluentd beside your container
- Filter, parse and enrich before sending
- Fan out to several destinations at once
- Needed for third-party destinations or splitting streams

THE TIEBREAKER

One destination and no processing means awslogs. Several destinations, or filtering before it is charged for, means FireLens.

EKS: one add-on does most of it

- The **Amazon CloudWatch Observability EKS add-on** deploys the **CloudWatch agent as a DaemonSet** and **Fluent Bit** for container logs.
- Agent handles metrics and Container Insights. **Fluent Bit handles the log lines.** Two components, two jobs.
- Permissions come from **EKS Pod Identity** or IRSA, not from the node's instance profile.
- Windows nodes need add-on **version 1.5.0 or later**, and Application Signals is not supported on Windows there.

> ON THE EXAM
>
> DaemonSet means one agent per node. That is how you get node-level metrics that a per-pod sidecar could never see.

Where the logs come from, by compute type

COMPUTE	HOW LOG LINES REACH CLOUDWATCH LOGS
EC2	CloudWatch agent tailing files you name in its config
ECS on EC2	awslogs log driver, or FireLens sidecar
ECS on Fargate	Same: awslogs or FireLens. No host to install anything on
EKS	Fluent Bit, deployed by the Observability add-on
Lambda	Automatic. Nothing to install; stdout goes to a log group
VPC, ELB, Route 53 etc	Vended logs. Enable the feature; no agent involved

ON THE EXAM

On Fargate there is no host, so any answer involving installing an agent on the instance is wrong by construction.

Sensitive data: masking at ingest

- A **data protection policy** on the log group detects and masks sensitive fields as events arrive.
- Uses **managed data identifiers** for PII, PHI and financial data, so you do not write the patterns yourself.
- **It applies at ingest only.** Events already in the log group are not masked retroactively.
- Auditors can be granted **Unmask** permission separately, so the data is hidden rather than destroyed.

> ON THE EXAM
>
> "Card numbers appeared in the logs last month" cannot be fixed by a policy. The policy protects tomorrow, not yesterday.

Access, encryption and other accounts

- Log groups are encrypted at rest by default. Associate a **KMS customer managed key** when you need to control the key.
- The **key policy must allow the CloudWatch Logs service principal**, or association fails and ingestion stops.
- **Cross-account observability** from module one covers logs too: search and query other accounts' log groups from the monitoring account.
- A **resource policy** on the log group is what lets another service put events into it. One policy per log group.

> ON THE EXAM
>
> A KMS-related logging outage is almost always the key policy, not the IAM policy of whoever is writing.

Two features worth knowing by name

- **Live Tail.** A streaming view of log events as they arrive, with filtering. For watching a deployment, not for investigating history.
- **Log anomaly detection.** Learns the normal pattern of a log group and surfaces new or unusual message types.
- **Neither works on Infrequent Access log groups.** Both are Standard-class features.
- Anomaly detection catches **a new stack trace nobody wrote a filter for**, which is the whole point.

> ON THE EXAM
>
> Metric filters find what you predicted. Log anomaly detection finds what you did not.

Making logs cheaper and faster to query

- **Field indexes.** Index high-cardinality fields such as request ID or user ID. Logs Insights then skips events that cannot match.
- Lower query cost and faster results. Currently **one field index policy per log group**.
- **Transformers** parse and restructure log events **at ingest**, so downstream filters and queries work on real fields.
- Transformers can be set **account-wide**, and a log-group-level transformer overrides the account one. No extra charge on Standard ingestion.

> ON THE EXAM
>
> Transform once at ingest instead of parsing in every query. The parsing was going to happen either way; this way you pay for it once.

A company runs 300 EC2 instances across six Auto Scaling groups. An engineer installed and configured the CloudWatch agent on each instance by hand over two weeks, using the configuration wizard.

Three months later, memory metrics are missing from about 40 percent of instances, several instances report a different set of log files from the rest, and the newest instances report nothing at all.

What should the company implement?

A Store one agent configuration in SSM Parameter Store and apply it with a State Manager association

B Use Run Command to reapply the wizard-generated configuration to all 300 instances

C Enable detailed monitoring on all six Auto Scaling groups

D Write a Lambda function on an EventBridge schedule that SSHes to each instance and checks the agent

◆ PAUSE AND DECIDE

What should the company implement?

A Parameter Store plus State Manager

One configuration, re-applied on a schedule, and automatically applied to instances that launch later. Fixes drift and new instances together.

B Run Command to reapply

The best wrong answer. It fixes today's 300 instances and does nothing for tomorrow's, so the drift returns immediately.

C Detailed monitoring

Changes how often existing metrics publish. Memory is not an EC2 metric at all, so there is nothing to publish more often.

D Lambda that connects to each instance

Rebuilds State Manager badly, with SSH keys, network paths and 300 connections to maintain. Maximum operational overhead.

THE SIGNAL

Two symptoms, two requirements: drift on existing instances, and nothing at all on new ones. Only a desired-state mechanism fixes both.

A security team enables a data protection policy with the credit card managed data identifier on a payments application's log group. They confirm new log events show masked card numbers.

An auditor then reports that log events from the previous six weeks still contain full card numbers in plain text, and asks the team to explain why the control did not work.

What is the correct explanation and remedy?

A. The policy needs the Unmask permission removed before it takes effect

B. Masking applies at ingestion only; historical events must be deleted, and shortening retention or recreating the log group is the remedy

C. The log group is in the Infrequent Access class, which does not support data protection

D. A second data protection policy must be applied at the account level to cover historical data

◆ PAUSE AND DECIDE

What is the correct explanation and remedy?

A. Remove Unmask permission

Unmask controls who can view masked values. It has nothing to do with whether older events were ever masked.

B. Masking is at ingest; delete the history

Documented behaviour: events ingested before the policy existed are not masked. Only removing them clears the exposure.

C. Infrequent Access does not support it

Was true until March 2026. Infrequent Access now supports data protection, and the masking on new events proves the class is not the issue.

D. An account-level policy for history

Account-level policies exist and are useful, but they also apply at ingestion. Scope is not the same as time.

THE SIGNAL

The control worked exactly as designed. It protects everything ingested after it was created, and nothing before.

A CloudOps engineer moves an application's log group to the Infrequent Access log class to reduce ingestion cost. The application runs on ECS Fargate and uses the awslogs driver.

After the change, a metric filter that counted OutOfMemoryError occurrences stops producing data points, and the alarm built on it moves to INSUFFICIENT_DATA. Queries in Logs Insights still work normally.

Why did this happen, and what is the fix?

A Fargate does not support metric filters; move the workload to ECS on EC2

B Infrequent Access does not support metric filters. Create a new Standard log group and point the task definition at it

C Change the log group's class back to Standard in the console

D The alarm needs its missing data treatment set to breaching

◆ PAUSE AND DECIDE

Why did this happen, and what is the fix?

A Fargate does not support metric filters

Invented limitation. Metric filters are a property of the log group and do not care what wrote to it.

B New Standard log group, repoint the task

Metric filters are Standard-only, and the class cannot be changed after creation. A new log group is the only route.

C Change the class back

The genuinely tempting one, and the trap. Log class is fixed at creation; there is no console option to change it.

D Missing data as breaching

Would make the alarm fire constantly while the underlying metric no longer exists. It hides the fault rather than fixing it.

THE SIGNAL

Logs Insights still working is the clue that narrows it. Infrequent Access kept querying and lost metric filters, which is exactly the observed behaviour.

> A team runs microservices on Amazon EKS. They need node-level and pod-level metrics in Container Insights, container log lines in CloudWatch Logs, and they must not attach broad CloudWatch permissions to the node instance profile, because every pod on a node would inherit them.
>
> The cluster runs Linux nodes only. The team wants the least operational overhead.

What should the team deploy?

A. The Amazon CloudWatch Observability EKS add-on, with permissions granted through EKS Pod Identity

B. The CloudWatch agent as a sidecar container in every pod, using the node instance profile

C. The awslogs log driver on each pod specification, plus detailed monitoring on the nodes

D. A FireLens sidecar in every pod, with CloudWatchAgentServerPolicy on the node role

◆ PAUSE AND DECIDE

What should the team deploy?

A Observability add-on with Pod Identity

One managed add-on gives the agent DaemonSet and Fluent Bit. Pod Identity scopes permissions to the service account, not the node.

B Agent sidecar per pod, node profile

A sidecar in every pod cannot see node-level metrics, and it uses exactly the node-wide permissions the team ruled out.

C awslogs driver on pods

awslogs is an ECS task definition setting. It is not how Kubernetes pods ship logs, and it produces no Container Insights metrics.

D FireLens sidecar, node role policy

FireLens is ECS terminology, it moves only logs and no metrics, and it still puts the permissions on the node role.

THE SIGNAL

The permissions clause is the discriminator. Every option except A grants CloudWatch access to the whole node, which the scenario explicitly forbids.

The five things to carry forward

- **Config in Parameter Store, pushed by State Manager.** Run Command fixes today; State Manager keeps it true.
- **Retention defaults to never expire**, and the allowed values are a fixed list. Ingestion and storage bill separately.
- **Log class is fixed at creation.** Infrequent Access loses metric filters, subscription filters, Live Tail and anomaly detection.
- **ECS is awslogs or FireLens. EKS is the Observability add-on:** agent DaemonSet for metrics, Fluent Bit for logs.
- **Masking applies at ingest only.** It protects tomorrow, never yesterday.

> ON THE EXAM
>
> Next: reading logs at scale - metric filters, subscription filters and Logs Insights. This module got the logs in; that one gets answers out.